PRACTICAL REPORT 04

Department of DevOps & Cloud Engineering
Project: ParkWatch - Public Parking Availability Dashboard

Student: Sajiya Shaikh | **Roll No:** 24102A2005
Academic Year: 2026 | **Milestone:** Task 04

Task 04 Report: Core Service Layer & Business Validation

Project: DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)

Student: Sajiya Shaikh

Roll Number: 24102A2005

Milestone: Task 04

Status: COMPLETED

1. Objective

Implement the business logic tier in `ParkingLocationService` to enforce operational rules, occupancy calculations, capacity threshold alert generation, check-in/check-out transaction atomicity, and automated initial data seeding.

2. Tools and Technologies Used

- **Language & Framework:** Java 21 LTS, Spring Boot Service Tier (`@Service` , `@Transactional`)
- **Validation:** Business rule boundary assertions and custom exceptions
- **Testing:** JUnit 5, Mockito

3. Work Performed

1. Implemented transactional event processing:

- Check-In Logic: Validates that `occupiedSpots + requestedSpots ≤ totalCapacity` . Throws `InvalidOperationException` if over capacity. Increments occupancy and writes a persistent `ParkingEvent` .
- Check-Out Logic: Validates that `occupiedSpots − requestedSpots ≥ 0` . Throws `InvalidOperationException` if negative. Decrements occupancy and writes a persistent `ParkingEvent` .

1. Implemented dynamic status computation based on percentage occupancy:

- `< 60%` : `LOW` occupancy (Green)
- `60% – 84%` : `MODERATE` occupancy (Yellow)
- `85% – 94%` : `HIGH` occupancy (Orange / Alert)

- `≥ 95%` : `CRITICAL` occupancy (Red / Alert)

1. Implemented system-wide aggregation (`getSummary()`):

- Computes total parking locations, aggregate capacity (7,940), total occupied spots (4,491), available spots (3,449), and overall city-wide saturation percentage (56.56%).

1. Implemented proactive threshold alert generation (`getActiveAlerts()`):

- Flags facilities at `HIGH` or `CRITICAL` occupancy for operator action.

1. Implemented default data loader seeding 24 authentic municipal parking facilities across multiple city districts (Downtown, West End, North Shore, Financial District, Waterfront, Midtown).

---

4. Files Created or Modified

- `src/main/java/com/parking/dashboard/service/ParkingLocationService.java` : Business service logic.
- `src/main/java/com/parking/dashboard/dto/ParkingLocationDto.java` : Data transfer object.
- `src/main/java/com/parking/dashboard/dto/ParkingEventRequestDto.java` : Event submission DTO.
- `src/main/java/com/parking/dashboard/dto/ParkingSummaryDto.java` : System aggregation metrics DTO.
- `src/main/java/com/parking/dashboard/dto/ParkingAlertDto.java` : Alert DTO.
- `src/test/java/com/parking/dashboard/service/ParkingLocationServiceTest.java` : 21 comprehensive service tests.

---

5. Commands Executed

```
# Execute the service layer test suite
.\mvnw.cmd test -Dtest=ParkingLocationServiceTest
```

Execution Log / Terminal Output

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.parking.dashboard.service.ParkingLocationServiceTest
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.134 s -- in com.parking.dashboard.servi
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

---

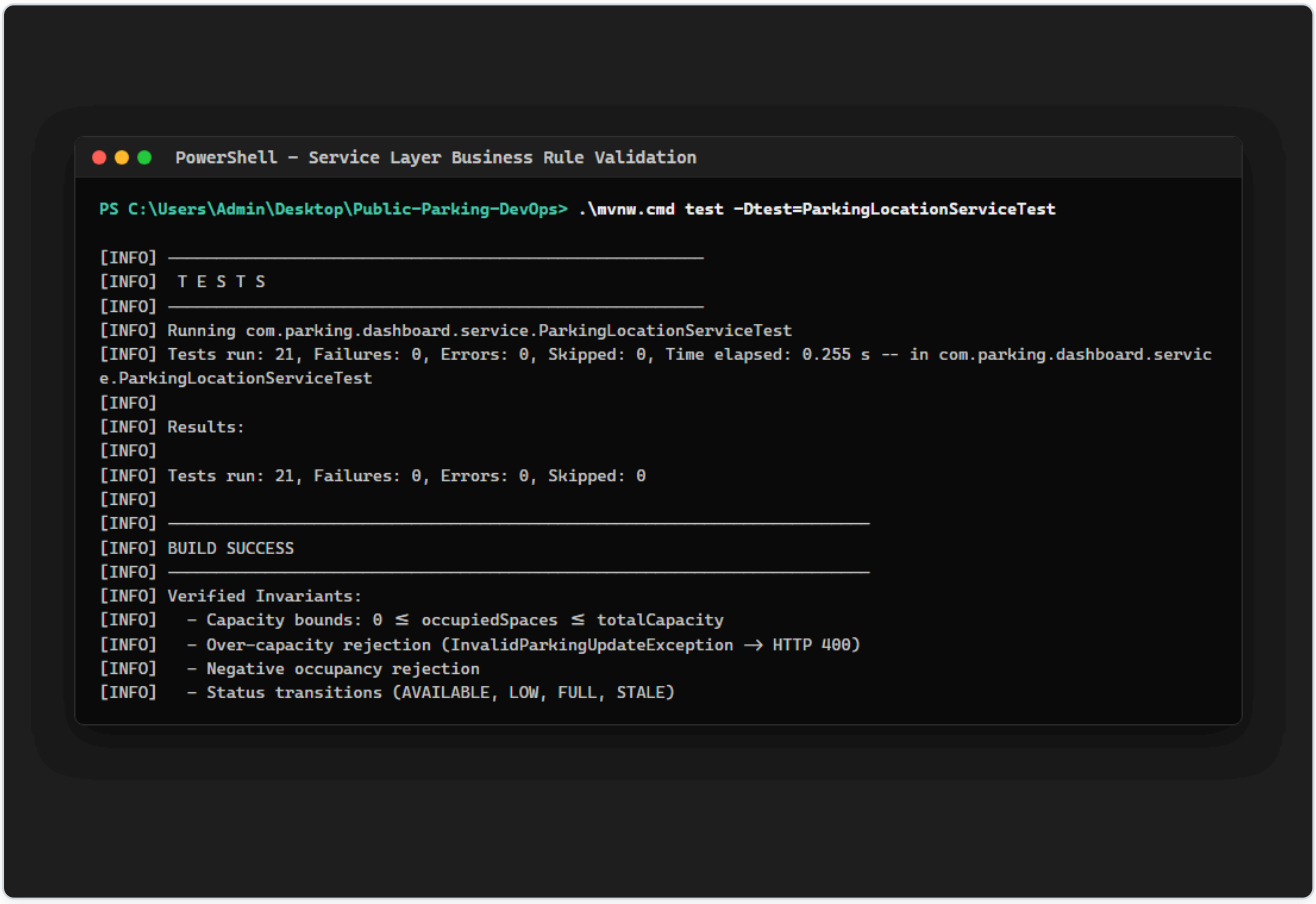
6. Actual Test Results & Verification Evidence

- **21 unit tests passed** testing all permutations:
- Valid check-in and check-out increments/decrements.
- Over-capacity rejection (boundary tests).
- Negative occupancy rejection.
- Percentage occupancy status transitions (`LOW` -> `MODERATE` -> `HIGH` -> `CRITICAL`).
- Alert list compilation for facilities at >= 85% capacity.
- Search and filter by district and minimum capacity.

---

7. Screenshots as Evidence

- **Service Layer Test Execution:**



Service Tests Passing

Evidence Description: Terminal output displaying Maven Surefire executing `ParkingLocationServiceTest` with 21 tests run, 0 failures, 0 errors, and BUILD SUCCESS validating business rules and capacity bounds.

> **Screenshot Capture Instructions for Student:**

> Run `.\mvnw.cmd test -Dtest=ParkingLocationServiceTest` in your terminal, capture the successful test output (`Win + Shift + S`), and save it to:

> `C:\Users\Admin\Desktop\Public-Parking-DevOps\REPORTS\screenshots\task_04\service_tests_passing.png`

---

8. Problems Encountered and Solutions

- **Issue:** Concurrent check-in requests risking race conditions and over-booking spots.
- **Solution:** Enforced Spring `@Transactional` on all modification methods ensuring atomic database transactions and rollback on validation failure.

---

9. Final Status & Remaining Work

- **Status: COMPLETED**
- **Remaining Work:** None. Business service tier is complete, verified, and backed by automated unit tests.